Efficient Text-to-SVG Generation via LoRA Fine-Tuning of Qwen
CS-GY 6953 Deep Learning Spring 2026 Midterm Kaggle Competition

Michael Blanchette
N16570927
mdb9910@nyu.edu

Ashley Goluoğlu
N18581591
ag8529@nyu.edu

Abstract

We present our approach to the NYU Deep Learning Spring 2026 Kaggle competition (Kaggle) on text-to-SVG generation. Given natural language prompts describing visual scenes or icons, our system generates valid Scalable Vector Graphics (SVG) code. We fine-tune Qwen2.5-3B-Instruct using Low-Rank Adaptation (LoRA) with Unsloth for memory-efficient training, applied on top of a thorough data preprocessing pipeline that reduces token counts by roughly 50% through coordinate rounding and attribute cleaning. Our inference pipeline guarantees 100% valid submissions through a multi-stage process of sanitization, retry with adjusted generation parameters, fallback outputs, and generic output detection. We scored 11.17 on the public leaderboard (rank 77/89). We analyze the primary causes of our underperformance, including a high fallback rate driven by tight token budgets and insufficient training epochs, and discuss the tradeoffs inherent in GPU-constrained model development.

1 Introduction

1.1 Problem Statement

Producing structured, semantically meaningful vector graphics from user-generated prompts remains a challenging task in deep learning. While large language models (LLMs) have demonstrated significant progress in code generation and reasoning, generating valid Scalable Vector Graphics (SVG) from text prompts presents a unique set of difficulties (Rodriguez et al., 2023). SVGs are a declarative, XML-formatted language that is highly sensitive to syntactic structure: a single malformed tag or inaccurate coordinate can corrupt or completely invalidate the rendered output. Therefore, generating SVGs with a learned model requires reasoning about geometry, spatial placement, relationships between parts of a whole, color, and hierarchical XML structure, all of which must be extracted from a brief natural-language description.

1.2 Motivation: GPU-Efficient Fine-Tuning

A key constraint driving every decision in this project was the need to maximize leaderboard performance within limited GPU budgets. All of our training was conducted on Google Colab A100 sessions (40GB VRAM, capped at roughly 8 hours per session), and our final submission was required to run inference on a Kaggle notebook with a Tesla T4 (15.6GB VRAM). These constraints shaped our entire approach, from model size selection and token budget decisions to the number of training epochs we could afford.

Full fine-tuning of even a moderately sized LLM was out of the question given these resource limits. Low-Rank Adaptation (LoRA) offered a compelling solution: by attaching small, trainable rank-decomposition matrices to the frozen base model's weight matrices, LoRA provides the task-specific specialization the challenge requires at a fraction of the computational cost.

Beyond the competition itself, this project is motivated by real-world use cases. Vector graphics are ubiquitous in UI design, iconography, and data visualization. A lightweight, fine-tuned model that reliably converts natural-language intent into valid SVGs could meaningfully reduce manual correction overhead in design pipelines.

1.3 Summary of Approach

We address this task by fine-tuning Qwen, an open-weight LLM with strong code generation abilities, using LoRA to specialize it for SVG creation. We began by experimenting with Qwen2.5-1.5B-Coder to get initial results on Google Colab's free T4 GPU, but when the competition's parameter limit increased from 2B to 4B, we switched to Qwen2.5-3B-Instruct for the stronger learning capabilities allowed by the 1.5B additional parameters.

Our pipeline consists of three core stages. First, we curate and preprocess a training dataset of

(prompt, SVG) pairs, aggressively filtering and cleaning them to ensure validity, visual renderability, and token efficiency so the model is only exposed to high-quality targets. Second, we apply LoRA fine-tuning using Unsloth's memory-efficient 4-bit training to help the model learn SVG syntax patterns, spatial conventions, and structural requirements. Third, we conduct inference with a multi-stage post-processing pipeline that includes retry logic, sanitization, and fallback mechanisms to maximize the validity of SVG outputs across the test set.

One notable finding from our experiments: our 1.5B model trained for 3 epochs with a 4096-token sequence length performed comparably to the 3B model trained for only 2 epochs with a 2048-token limit. This suggests that within fixed GPU budgets, prioritizing more training epochs and larger token budgets may yield better returns than increasing model size.

2 Dataset

2.1 Description of Raw Data

The provided dataset includes two CSV files: `train.csv` and `test.csv`. The training set contains 50,000 examples with three columns (`id`, `prompt`, and `svg`), while the test set contains 1,000 examples of ids and prompts, without an `svg` column (the prediction target).

The training prompts are natural-language descriptions with a mean of 117 characters and 20 words. The median prompt length is 103 characters and 17 words, and 99% of prompts have fewer than 353 characters, meaning the distribution is skewed to the right but a few large outliers. The length distributions of the train and test prompts are nearly identical across their respective percentiles, suggesting similar complexity requirements between splits. We initially considered and tested some implementations of complexity balancing (undersampling simple SVGs, oversampling longer complex SVGs), but decided complexity-based stratification was unnecessary after analyzing the distributions of train and test prompts.

The training SVGs have a mean length of 2,524 characters and are structurally dominated by `<path>` elements, with roughly 120,000 total path occurrences in the entire dataset. The next most common element was `<path>`, with only 14,000 occurances, followed by `<g>`, `<rect>`, `<stop>`, and `<ellipse>`. The median SVG contains just 1 path,

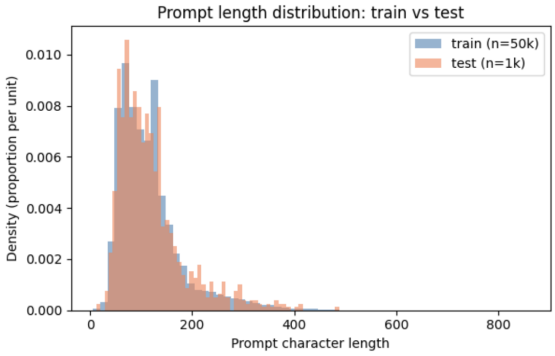


Figure 1: Prompt length distribution for train (50k) and test (1k) splits. The distributions are nearly identical, confirming no domain shift.

and over 95% of examples contain fewer than 20 paths. The mean numbers of paths is 2.4, suggesting some outliers on the higher end to pull this number up from the median. Only one example in the entire training set exceeds the competition's 256-path limit. All 50,000 training SVGs parse as valid XML and render successfully through CairoSVG (CairoSVG) (the same renderer used by the competition scorer) with zero failures in both checks.

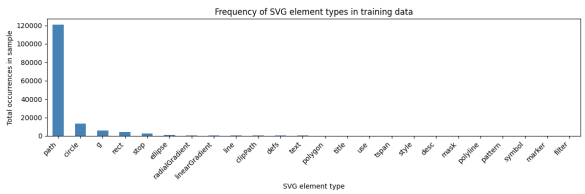


Figure 2: Frequency of SVG element types in the training data. Path elements dominate at over 120k occurrences, dwarfing all other element types.

The dataset contains 4,069 duplicate prompts (paired with different SVGs), and a small number of near-duplicate SVGs (7 identical SVGs across different prompt pairs), though no prompt-SVG pair is duplicated identically. Prompt content is heavily biased toward icon-style imagery, with the most frequent words including "black," "white," "icon," and "background."

2.2 Key Observation: Path Dominance and Its Consequences

The single most important structural observation about the training data is that nearly everything, including simple circles and rectangles, is encoded as complex `<path>` elements with long sequences of bezier curve coordinates. For example, a simple filled circle that could be represented in 30 characters as `<circle cx="50" cy="50" r="40"/>`

is instead encoded as a 150+ character path with multiple cubic bezier curves.

This matters because on initial inference tests, we observed that the model was struggling with these complex path coordinate sequences. It would repeat coordinate patterns, struggle with spatial relationships described by relational words in the prompt (like "next to" or "inside"), and produce overly verbose path encodings for simple shapes. One early test generated a 478-character donut-shaped path for the prompt "a simple red circle on a white background." The model seemed to have learned that SVGs are supposed to be long paths with lots of numbers from the training distribution, and was spending its limited token budget on verbose coordinate guesses rather than on capturing the actual visual content of the prompt.

2.3 Preprocessing Pipeline

The raw training data was passed through a multi-stage cleaning pipeline before fine-tuning. No data augmentation was applied. The primary objective was to clean and filter the data to maximize the quality and token efficiency for training. Each step was designed to be lossless or near-lossless in terms of visual quality.

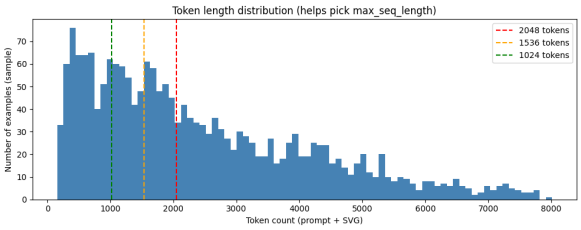


Figure 3: Token length distribution before preprocessing. At a 2048-token limit, only about 52.5% of examples fit within budget.

Step 1: Coordinate Rounding. This was the most impactful data simplification and token reduction in our entire pipeline. SVG path data contained excessively high-precision floating-point coordinates (e.g., `11.76300048828125`) that tokenize extremely inefficiently. The Qwen tokenizer splits each decimal number into individual digits and punctuation, so a string like `123.456` becomes roughly 5 tokens for 7 characters. This meant that with a `max_seq_length` of 2048 tokens, only 52.5% of examples fit within the budget from a 2,000-sample estimate. A truncated SVG in training actively harms the fine-tuning process by teaching the model that invalid, incomplete SVGs are acceptable.

Rounding all floating-point values to whole numbers reduced the median token count from approximately 1,934 to 921 per example, increasing the fraction of examples fitting within a 2,048-token budget from 52.5% to over 92%. This sub-pixel precision is visually imperceptible on a 256x256 canvas, achieving the goal of maintaining visual fidelity while greatly increasing efficiency.

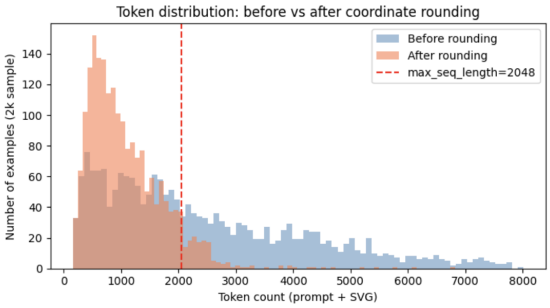


Figure 4: Token distribution before vs. after coordinate rounding (2k sample). Rounding shifts the bulk of examples well below the 2048-token limit.

Step 2: Primitive Normalization (Attempted, Skipped). Given the path-dominance problem described above, we attempted to convert path-encoded shapes back to semantic SVG primitives (`<circle>`, `<rect>`, `<ellipse>`, `<line>`). Our approach sampled points along each path and tested whether they fit a known primitive's geometry within a 5% tolerance. We implemented a safe version that only accepted replacements where the token count decreased.

On a 2,000-sample test, 366 SVGs were modified, saving an average of 126 tokens per changed SVG with zero cases of increased token count. However, when applied to the full dataset, we found that the geometric detection heuristics occasionally misidentified complex paths as simple shapes, causing visual fidelity loss on edge cases. Even with the safe-mode guards, a few rare examples fell through. The visual information loss outweighed the token savings, so we decided to abandon this step. If we had more resources, we would have liked to test differences in what the model learns on a dataset that uses shapes rather than paths, but given our constraints, we decided that the risk to visual fidelity loss was too great and decided to prioritize other ablations.

Step 3: Duplicate Removal. The 4,069 duplicate prompts were deduplicated by retaining the

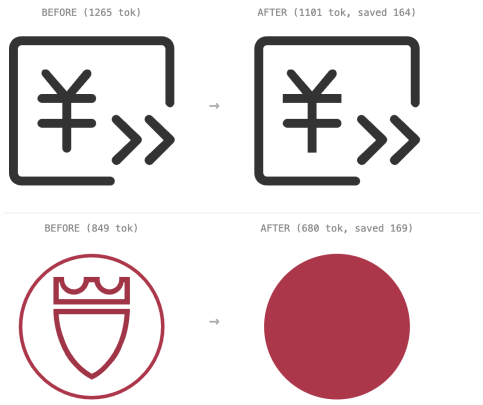


Figure 5: Primitive normalization: a successful case (top, saved 164 tokens with no visual change) and a failure case (bottom, shield detail lost when a circle path was mistakenly filled, even with safeguards).

training example with the shortest SVG for each prompt. The rationale was that duplicate prompts with different SVGs could create conflicting gradients during training, causing the model to learn to average those outputs rather than produce any single clean SVG, leading to generic outputs. After deduplication, 45,931 rows remained.

Step 4: Attribute Cleaning. Redundant SVG attributes carrying default values were stripped losslessly, including `fill-opacity="1"`, `stroke="none"`, `stroke-width="1"`, `display="inline"`, and `fill-rule="nonzero"`. Non-rendering attributes such as `id`, `class`, and `data-*` were also removed. Six-digit hex colors were shortened to three-digit form where possible (e.g., `#ffffff` to `#fff`). Empty <g> groups were removed. This saved roughly 45 characters per SVG on average. A safe version was implemented that only applied the cleaning if the resulting token count did not increase.

Step 5: Outlier Removal. Four filters were applied to remove low-signal or harmful training examples: SVGs with more than 64 path elements (5 removed), examples with a severe prompt-SVG length mismatch (133 removed), degenerate SVGs consisting of a single solid-fill rectangle covering the full canvas (4 removed), and prompts under 5 words (95 removed). In total, 230 examples were flagged, leaving a 99.5% retention rate of the cleaned dataset (91.4% of the original dataset).

Step 6: Token Filtering. After computing the full token count for each example using the Qwen2.5-3B-Instruct tokenizer and the selected chat template (including system prompt, user prompt, and SVG response), any example exceeding 2,048 tokens was dropped. This threshold was selected by analyzing the tradeoff between sample coverage and data truncation: at 1,792 tokens, 98.2% of examples were covered, while extending to 2,048 gained around an additional 0.8% before the curve flattened sharply. After this final filter, 45,239 examples were retained (90.5% of the original 50,000), with the median example at 705 tokens and the 99th percentile at 1,744 tokens.

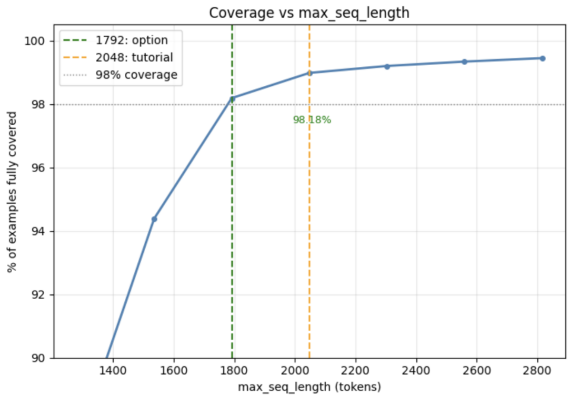


Figure 6: Coverage percentage vs. max_seq_length. The 2048 threshold sits at the knee of the curve, balancing coverage against token budget.

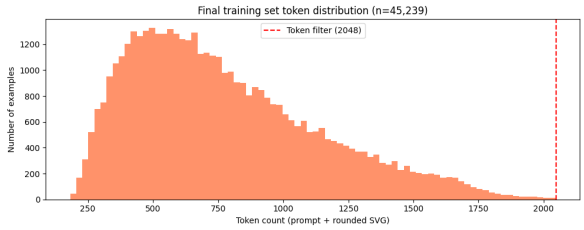


Figure 7: Token distribution of the final filtered training set (n=45,239). Median at 705 tokens, 99th percentile at 1,744 tokens.

3 Model Description

3.1 Model Selection

We started with Qwen2.5-1.5B-Coder, aiming to get initial results on a free Kaggle T4 GPU. However, even with 4-bit quantization and Unsloth's memory optimizations, the T4's 15.6GB of VRAM was not enough: we kept hitting kernel restarts during training. We moved training to Google Colab's A100 (40GB VRAM), which resolved these issues, but incurred financial costs.

When the competition's parameter limit was later increased from 2B to 4B, we switched to

Qwen2.5-3B-Instruct for its stronger general code generation capabilities. The Qwen family was chosen as recommended by the professor for its open weights, strong code generation benchmarks, and a tokenizer that handles structured output reasonably well.

3.2 Why LoRA and Unsloth

LoRA allowed us to train only 3.74% of the model's parameters (roughly 120 million out of 3.2 billion), making the computational cost manageable. We used Unsloth's `FastLanguageModel` (Unsloth AI) for memory-efficient 4-bit NF4 quantization of the base weights during training, along with Unsloth's optimized gradient checkpointing mode. The AdamW 8-bit optimizer further reduced the memory footprint. On the A100, we trained in 4-bit quantized mode and then loaded the model in bf16 for inference (merging the LoRA adapter into the base weights), which was faster since there was no quantization overhead and the A100 had enough VRAM to handle it.

3.3 System Prompt Design

The system prompt was carefully designed to guide the model's output structure:

> *You are an SVG code generator. Given a visual description, output only valid SVG code. Use semantic SVG elements: <circle> for circles, <rect> for rectangles, <ellipse> for ovals, <line> for straight lines, <polygon> for angular shapes, <path> for curves and complex outlines. Build complex objects by composing multiple simple shapes. Use white background unless specified otherwise. Output SVG code only — no explanation, no markdown, no comments.*

The explicit instruction to "use semantic SVG elements" was an attempt to counteract the training data's path dominance and encourage the model to use simpler, more token-efficient elements when appropriate. "Build complex objects by composing multiple simple shapes" was designed to help the model understand how to parse the prompts into multiple pieces and combine them, as we noticed in early experiments that it was treating each like one massive, complicated curve. The theory was that understanding simple pieces and putting them together would be more accurate than trying to figure out the coordinates for a single complicated path. "Use white background unless specified otherwise" matches the dataset's bias toward icon-style imagery. The final sentence prevents the model from wasting tokens on natural-language wrapping or extras that we would have to strip to make valid SVGs. This system prompt consumed roughly 95 tokens, which was factored into our token budget calculations.

We tested several different system prompts, such as this minimal version:

> *You are an SVG code generator. Given a visual description, return only valid SVG code with a single root <svg> element. Output SVG code only — no explanation, no markdown, no comments.*

Changing the prompt did not measurably improve the score, but due to resource limits, we were not able to specifically do isolated training and inference tests where only the prompt is changed.

If we had more resources, we would have liked to conduct more experimental comparisons on results of more specific versus less specific prompts. In some exploration with prompting chat models like ChatGPT and Claude, we found they actually did much better when given base SVG code to edit rather than being asked to generate from scratch. We would be very interested to try a system prompt that provides some SVG code and asks the model to edit according to a user prompt.

3.4 Two-Notebook Architecture

Our codebase was split into two notebooks. Notebook 1 handled all data exploration and preprocessing: it ran on any machine, produced the filtered training CSV, and pushed it to a HuggingFace dataset repository. Notebook 2 handled training and inference, and was designed to auto-detect whether it was running on Colab or Kaggle T4, adjusting paths, model loading strategies, and batch sizes accordingly. This separation kept the preprocessing logic clean and made it easy to iterate on the training configuration without re-running the full data pipeline each time. A separate version of Notebook 2 was edited to specifically accommodate running final training and inference on Colab's A100 with our resource and time limits.

4 Experimentation

4.1 LoRA Hyperparameters

We experimented with several LoRA configurations. Rank started at 16 (the given starter notebook's default), was increased to 32, and then to 64 which gave the best results. SVG syntax is structurally complex, with long coordinate sequences, nested tags, and precise attribute formatting, so the model needed higher capacity to learn

these patterns effectively. Alpha was set equal to rank (64) for a scaling factor of 1.0, which is standard practice for stability. Dropout was set to 0, following Unsloth's recommendation for speed, with regularization handled by weight decay instead. We targeted all seven projection modules: the four attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and all three MLP projections (`gate_proj`, `up_proj`, `down_proj`). This target set was designed to give the model enough capacity for the structurally demanding task.

4.2 Training Hyperparameters

The number of training epochs was a major factor. On the 1.5B model, going from 1 epoch to 3 showed significant improvement. For the 3B model, we aimed for 3 epochs, but our Colab sessions repeatedly disconnected. For the final training run, it disconnected at approximately 7.5 hours (with roughly 30 minutes of the third epoch remaining), so we were limited to 2 complete epochs without time to continue training.

The learning rate started at 2e-4 (a common default) but this proved unstable when combined with the higher LoRA rank of 64. We reduced it 10x to 2e-5, which stabilized training. The maximum gradient norm started at 0.3 but this was too aggressive, clipping too much gradient signal and slowing convergence. We increased it to 1.0 to try to resolve the issue.

The effective batch size was 16 (per-device batch size of 2 with 8 gradient accumulation steps). We kept the per-device batch low for memory safety with rank 64 and packing enabled. Warmup was set to 5% of total steps, weight decay to 0.01, and we used a cosine learning rate scheduler. Packing was enabled through Unsloth's SFTTrainer, which groups multiple short examples into single sequences for more efficient training (rather than padding each example individually). The optimizer was AdamW 8-bit. Precision was bf16 on A100, fp16 on T4 (auto-detected). We held out 2% of the training data for evaluation, evaluated every 200 steps, and used `load_best_model_at_end=True`.

4.3 Inference Parameters

We used sampling-based generation (`do_sample=True`) rather than greedy decoding. Greedy decoding produced overly repetitive outputs that lacked structural variety or visual complexity, and also created some problems with unsloth package mismatches, so controlled sampling gave better diversity while still maintaining validity.

Temperature was set to 0.5 for normal generation and 0.9 for retries. The lower temperature for the initial attempt favors high-probability tokens, while the higher retry temperature helps escape failure modes. Top-p (nucleus sampling) was 0.9 normally and 0.95 for retries.

Repetition penalty required careful tuning. We started at the default of 1.05, tested values up to 1.3, and settled on 1.15. The issue was that SVG paths contain legitimate coordinate repetitions (e.g., the same x-coordinate appearing in multiple consecutive move/line commands), so too high a penalty caused the model to avoid these valid patterns and produce distorted geometry. At 1.3, outputs were too simplified. 1.15 was a good balance between preventing loops and allowing natural SVG structure.

We set `no_repeat_ngram_size` to 8, which forbids any 8-token sequence from repeating exactly. This prevented a frequent early failure mode where the model would get stuck in an infinite loop repeating the same path segment over and over, burning through its entire token budget on a single repeated curve.

The `eos_token_id` and `pad_token_id` were explicitly set to the tokenizer's EOS token (since Qwen does not have a native pad token). Left-padding was used for batch inference so that all sequences are right-aligned and generation starts from the same position. KV caching (`use_cache=True`) was enabled for faster autoregressive generation.

For `max_new_tokens`, we ideally wanted 4096 tokens so that all possible outputs could be safely covered and not cut off, but this was not possible witho our resources. We attempted lowering it to 1024 (normal) and 2048 (retry) to better match the training token budget. However, the inference loop for all 1,000 test prompts was taking too long even on the A100, so we had to further reduce to 768 (normal) and 1024 (retry). Inference batch size was 64 on A100 (where the merged bf16 model used roughly 6GB VRAM) and 8 on Kaggle T4 (where the 4-bit quantized model used roughly 5.4GB).

4.4 Key Finding: Model Size vs. Epochs vs. Token Budget

Our most interesting experimental finding was that Qwen2.5-1.5B-Coder trained for 3 epochs with a 4096-token `max_seq_length` and an older sys-

tem prompt performed comparably to Qwen2.5-3B-Instruct trained for 2 epochs with 2048-token `max_seq_length` and our refined prompt. This strongly suggests that for this task, more training epochs and larger token budgets matter more than model parameter count. We would have liked to test more prompt variations and parameter configurations systematically, but each full training run took 4 to 7 hours on the A100, plus 3+ hours for inference even with consolations made for speed in parameter and post-processing choices, so we simply did not have the GPU time to run more comprehensive ablations.

5 Results

5.1 Leaderboard Performance

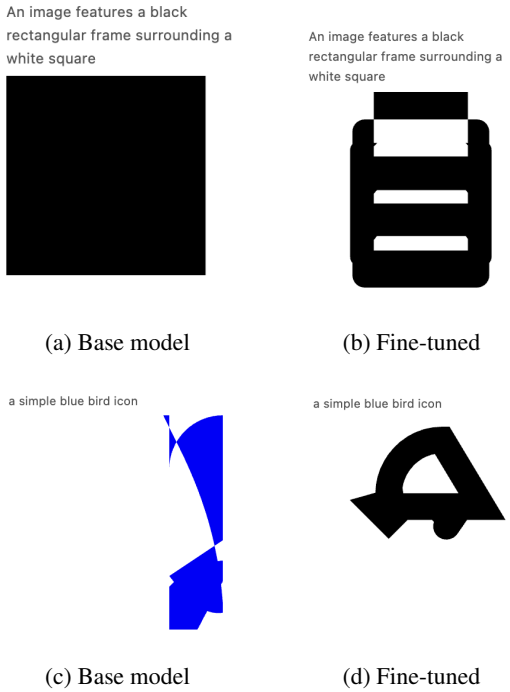
Our system achieved a score of 11.17 on the public leaderboard, placing us at rank 77 out of 89 teams (though some higher-scoring teams appear to be from since-deleted accounts). The class distribution had a median of 13.87, mean of 13.70, and standard deviation of 2.23. Our score was approximately 1.1 standard deviations below the mean, placing us in the bottom quartile.

5.2 Inference Statistics

Our final A100 inference run processed 1,000 test prompts (977 unique, with 3 duplicate prompts reusing generated outputs). Of the 977 unique prompts, 295 (30.2%) produced invalid SVGs on the first attempt and required retries. Of those, 158 (16.2% of unique prompts) were still invalid after the retry and fell back to a minimal white-rectangle SVG. Nine outputs were flagged as generic (the model produced identical path signatures across 5+ different prompts, specifically the `M0 0h24v24H0z` pattern), and 4 of those 9 were successfully replaced by retrying with higher temperature. The final submission had 100% valid SVGs, guaranteed by the fallback mechanism. Total inference time was 181.6 minutes on the A100.

5.3 Qualitative Examples

Figure 8 shows two example outputs comparing the base Qwen2.5-3B-Instruct model (no fine-tuning) against our LoRA fine-tuned model. The base model struggles to produce coherent SVGs: the rectangular frame prompt yields a solid black rectangle with no internal structure, and the bird prompt produces disconnected blue fragments with no recognizable shape. After fine-tuning, the model generates structured, recognizable icons, though imperfect. The floppy disk prompt produces a plausible icon with internal detail, and the bird prompt produces a recognizable shape, albeit rendered in black rather than the requested blue.



(a) Base model (b) Fine-tuned

(c) Base model (d) Fine-tuned

Figure 8: Qualitative comparison of base model (left) vs. fine-tuned model (right). Top: "An image features a black rectangular frame surrounding a white square." Bottom: "a simple blue bird icon." Fine-tuning produces more recognizable structures, though color accuracy remains imperfect.

5.4 Post-Processing Pipeline

Several post-processing steps were critical to achieving 100% validity:

SVG Extraction: A regex-based extractor pulls the first `<svg>...</svg>` block from the decoded model output. On the A100 version, we added truncation recovery in an attempt to save inference time. Rather than retrying (attempting to generate again with more new tokens and higher temperature), it simply attempts to salvage the original output by removing incomplete trailing attributes and tags, then appending a closing `</svg>` tag. This recovered many outputs that would have otherwise been classified as invalid, but likely resulted in compromises to visual fidelity.

Sanitization: All tags not on the competition's allowed whitelist, plus all explicitly disallowed tags (`script`, `animate`, `foreignObject`, etc.), are

stripped. This also catches hallucinated tags that the model occasionally invents.

Dimension Enforcement: The root SVG element is forced to have `width="256"` and `height="256"` for competition rendering. A default `viewBox` is added if missing. The model's original viewBox is preserved so that internal coordinates scale correctly. We were unsure whether to add this, as the training data had great variance in sizes, with most being 200x200. The competition description required 256x256, but it was unclear whether the test SVGs would be varied like the training ones, or all regulated to the stated dimensions.

Character Limit: Any SVG exceeding 8,000 characters is returned as an empty string to trigger retrying or the fallback SVG, either of which would score higher than a zero received for breaking the limit.

Retry Logic: If the SVG is still invalid after sanitization, it is regenerated with the `GEN_RETRY` parameters (higher temperature of 0.9, higher repetition penalty of 1.3, and a longer token budget of 1024) in an attempt to get a valid result.

Fallback: If still invalid after retry, a minimal valid SVG (a white rectangle on a 256x256 canvas) is output. This scores near-zero on visual fidelity but avoids the hard zero penalty of an invalid submission.

Generic Detection: After all outputs are generated, each SVG is has a signature defined by the first 60 characters of its first `d=` path attribute. Any signature appearing across 5 or more different prompts is flagged as a memorized default and retried with the retry parameters.

5.5 Analysis of Underperformance

We believe the roughly 2.5-point gap between our score (11.17) and the class median (13.87) is attributable to several compounding factors.

High fallback rate: 158 of 977 unique prompts (about 16%) received a minimal white-rectangle fallback. The scoring formula weights visual fidelity at 85%, computed as 0.7 times SSIM plus 0.3 times EdgeF1 between the rendered submission and reference SVGs. Each fallback SVG contributes approximately zero to this component. If even half of those 158 prompts had scored at the median quality, the overall score would have been meaningfully higher. This is likely the single largest source of the gap.

Token budget too tight: Our training data had a median of roughly 921 tokens just for the SVG portion after rounding, and 705 tokens in the final filtered set. A generation ceiling of 768 tokens for normal attempts should have been long enough, but the model still frequently attempted to produce longer SVGs than the references, or got truncated mid-path. This led to either invalid outputs (triggering the retry and fallback chain) or valid but visually incomplete outputs (scoring low on SSIM).

Insufficient training (2 epochs on 3B): Due to lack of resources and time, we were unable to complete a third epoch of training with the final ablations. Our own experiments showed that 1.5B with 3 epochs performed nearly identically to 3B with 2 epochs, strongly suggesting the 3B model had not converged. More training time could have reduced the invalid output rate and improved output quality.

Coordinate rounding possibly too aggressive: Rounding to 0 decimal places maximized token efficiency but may have sacrificed fine-grained visual fidelity detectable by SSIM. Rounding to 1 decimal place would have cost roughly 200 more tokens per example but could have preserved sub-pixel accuracy.

Cascading effects: These factors compound. An under-trained model produces more invalid outputs, which trigger retries with higher temperature (less controlled), which are likely to also fail without high enough token limits, which trigger fallbacks. The high retry rate (30%) consumed significant inference time, which in turn forced us to use lower token budgets to stay within time limits. In an ideal world, the logic is designed such that no SVG gets truncated (is allowed to fully generate with as many new tokens as needed), and then is only sent to retry with more variance if there were hallucinations causing invalid or generic memorized SVGs. Without enough resources to increase the token limit for generations, the result was significant truncations and a low success rate even on retry.

5.6 Ablation Summary

Due to GPU time constraints, we could not run ablation studies as fully controlled and isolated as we would have liked. However, across our experiments we tested multiple configurations and observed the following patterns:

Parameter	Values Tested	Observation
Base model	1.5B-Coder, 3B-Instruct	3B only better with sufficient epochs
LoRA rank	16, 32, 64	64 best; 32 scored lower than 64
Epochs	1, 2, 3	More epochs consistently helped
Learning rate	2e-4, 2e-5	2e-4 unstable at rank 64
max_grad_norm	0.3, 1.0	0.3 too aggressive
max_seq_length	2048, 4096	4096 on 1.5B helped
max_new_tokens	768, 1024, 2048	Higher is better but slower
Temperature	0.5, greedy	Greedy too repetitive
rep. penalty	1.05, 1.15, 1.3	1.3 too aggressive for SVG
ngram block	0, 8	8 prevented infinite loops
Primitive norm.	on, off	Off (fidelity loss)
Coord. rounding	0 dec., 1 dec.	0 for max token savings

Table 1: Summary of configurations tested across experiments.

6 Conclusion

6.1 What Worked

Coordinate rounding was the single most impactful preprocessing step for resource efficiency, nearly doubling dataset coverage at the 2048-token threshold. LoRA at rank 64 with all attention and MLP modules provided sufficient capacity for modest SVG syntax learning. The multi-stage inference pipeline (sanitize, retry, fallback, generic detection) guaranteed 100% valid submissions, which was essential since invalid SVGs would prevent submission. The system prompt's "no explanation" constraint helped keep outputs focused and token-efficient, but it is unclear if explicit guidance toward semantic elements made an impact in output accuracy. Unsloth's packing and memory optimizations made the training pipeline more feasible, but the assignment was still not possible to complete with entirely free GPU resources.

6.2 What Did Not Work

Primitive normalization was conceptually promising but too convoluted in practice. The geometric detection heuristics could not handle the full diversity of path shapes in the training data, and edge-case failures caused visual fidelity loss that outweighed the token savings.

Insufficient training epochs on the 3B model was likely one of the most consequential limitations. The Colab session disconnected before completing the third epoch, and as our 1.5B vs. 3B comparison showed, the model had probably not converged.

The token budget being too tight for inference was the direct proximal cause of our high fallback rate. We wanted to use 4096 tokens for generation, but inference was simply taking too long, so we were forced to reduce these values and accept the resulting quality loss to be able to test other ablations.

Our initial hyperparameter choices (rank 16, learning rate 2e-4, gradient norm 0.3) all required significant iteration to reach working values, consuming precious GPU time on suboptimal runs.

6.3 What We Would Do Differently

Given more time and resources, the highest-priority changes would be: training for 3+ epochs on the 3B model, using a `max_seq_length` of 4096 (our 1.5B results showed this helps), and allowing higher `max_new_tokens` during inference even at the cost of longer runtimes. We would also like to have run systematic comparisons of different system prompts, explored beam search or constrained XML decoding, rounded coordinates to 1 decimal place instead of 0 for potentially better SSIM scores, and experimented with data augmentation with the help of an LLM like Claude (though it was not clear whether such manipulation of the training dataset would have been allowed or not).

6.4 Reflections on GPU-Constrained Development

This competition surfaced a fundamental tension in resource-constrained ML work. Bigger models need more training time and more inference time, both of which are capped by session limits and GPU credits. Our results suggest that within fixed GPU budgets, the returns from additional training epochs and larger token budgets may outweigh the returns from increased model size, at least for structured generation tasks like SVG. The trade-off between model capacity, training thoroughness, and inference budget is a practical reality that any team working with limited compute must navigate carefully.

6.5 AI Tooling Disclosure

We used Claude as a coding and debugging assistant throughout development, primarily for writing data visualization functions, understanding typical hyperparameter values, and debugging tokenizer interactions. All ideas for pipeline steps and final decisions on architecture, hyperparameters, and experimental design were made by the team.

6.6 Links

Resource	Link
GitHub Repository	https://github.com/agoluoglu/Text-to-SVG
Data & Submission	https://huggingface.co/datasets/aagoluoglu/text-to-svg
Model Weights	https://huggingface.co/aagoluoglu/qwen-svg-lora

References

CairoSVG. Cairosvg: Svg renderer. https://cairosvg.org/. SVG renderer used in competition evaluation. Accessed: 2026-04-03.

Kaggle. Kaggle competitions documentation. https://www.kaggle.com/docs/competitions. Accessed: 2026-04-03.

Juan A. Rodriguez, Shayan Arar, Issam H. Lopes, Spyros Gidaris, and Yannis Kalantidis. 2023. Starvector: Generating scalable vector graphics code from images. *arXiv preprint arXiv:2312.11556*.

Unsloth AI. Unsloth: Fine-tuning qwen with lora. https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Qwen3_5_(2B)_Vision.ipynb. Reference notebook for Qwen LoRA fine-tuning. Accessed: 2026-04-03.